

TP 3 — Livraison progressive et observabilité : DevHub Campus SRE

Troisième et dernière partie du cours Kubernetes — Argo Rollouts, Prometheus, SLOs (vue plateforme)

Cours d'architecture logicielle & déploiement — ESGI 5 SRC

2025–2026

Table des matières

Avant-propos	1
Pourquoi la livraison progressive, et pourquoi maintenant	6
Le même geste, trois paradigmes	7
Architecture cible	9
PARTIE I — Observabilité avant tout	10
PARTIE II — Livraison progressive	17
PARTIE III — Synthèse et maîtrise	24
Annexe A — Pour aller plus loin	28
Annexe B — Pièges fréquents	29
Annexe C — Glossaire express	30

Avant-propos

Place de ce TP dans le cours Kubernetes

Ce polycopié est la **troisième et dernière partie** du cours Kubernetes d'ESGI 5 SRC. Il fait suite à :

- **TP 1 — SalleEnFrance** : provisionner un cluster, écrire des manifestes Kubernetes à la main, monter une pipeline CI/CD avec `kubectl apply`, sécuriser par mTLS. *Posture DevOps*.
- **TP 2 — DevHub Campus** : adopter GitOps avec ArgoCD, packager en Helm, ouvrir des environnements de preview par branche via `ApplicationSet`. *Posture développeur livreur*.
- **TP 3 — DevHub Campus SRE (ce document)** : livrer en mesurant l'impact, brancher Argo Rollouts sur Prometheus pour des canaries auto-promus ou auto-rollbackés, écrire des SLOs et un alerting gradé. *Posture SRE / plateforme*.

Vous n'avez pas besoin d'avoir terminé l'intégralité du TP 2 pour démarrer le TP 3 — un cluster ArgoCD opérationnel suffit. Le squelette fourni vous donne, le cas échéant, un point de départ équivalent à la fin du TP 2.

Contexte métier

Six mois ont passé. DevHub Campus — la plateforme interne que vous aviez déployée au TP 2 — est devenue le pivot quotidien des équipes pédagogiques de l'école. Les trois services métier (annuaire, planning, notif) sont sous ArgoCD, les développeurs poussent leurs branches, leurs previews apparaissent, ArgoCD réconcilie. Tout va bien.

Tout allait bien, jusqu'à la semaine dernière.

Mardi, 9h30. Un développeur de l'équipe planning fusionne une PR. L'image se construit, ArgoCD se synchronise sur main, le RollingUpdate propage les nouveaux pods. Aucune erreur dans l'UI ArgoCD : *Synced + Healthy*, tout est vert.

Sauf qu'à 9h31, les emplois du temps de 1200 étudiants se mettent à renvoyer des créneaux décalés d'une heure. Personne ne s'en rend compte avant 10h45 — quand un enseignant rate son cours. Rollback Git à 10h47. Convergence à 10h48. Mais le mal est fait.

Vendredi, 16h. Réunion d'équipe. Conclusion : « *ArgoCD nous dit que l'état du cluster correspond au Git. Il ne nous dit pas que le service fonctionne.* »

La direction technique mandate une nouvelle équipe : **l'équipe SRE plateforme**. Sa mission tient en une phrase : « *on ne déploie plus rien sans le mesurer. Et on ne promet rien qui dégrade les métriques.* »

Vous êtes cette équipe. Votre mission : instrumenter les services pour qu'ils émettent des métriques exploitables, monter la stack d'observabilité, et brancher la livraison progressive (canary, blue/green) sur des AnalysisTemplates qui décident automatiquement s'il faut promouvoir ou rollback.

Suite logique des TP 1 et TP 2

Vous changez de posture pour la troisième fois.

TP	Posture	Question centrale
TP 1 — SalleEnFrance	DevOps / SRE qui <i>provisionne</i>	« Comment monter une plateforme K8s reproductible ? »
TP 2 — DevHub Campus	Développeur qui <i>livre</i>	« Comment livrer sans toucher au cluster ? »
TP 3 — DevHub Campus SRE	SRE qui <i>protège la prod</i>	« Comment livrer sans casser ce qui marche ? »

Là où le TP 2 vous a fait *pousser plus vite*, le TP 3 vous fait **pousser plus sûr**. La rapidité de livraison sans filet de sécurité, c'est juste un raccourci vers l'incident. Cette idée — la *production-readiness* d'une chaîne GitOps — est le fil rouge du TP. Vous le retrouverez dans chaque étape, et vous devrez le restituer dans la synthèse finale.

Compétences visées

À la fin du TP, vous saurez :

1. Définir et défendre un SLO réaliste sur un service web, distinguer SLI, SLO, SLA, et calculer un *error budget*.
2. Instrumenter un service Node.js, Python ou Go avec le client Prometheus officiel pour exposer des métriques RED (Rate, Errors, Duration).
3. Installer kube-prometheus-stack via ArgoCD, comprendre la chaîne ServiceMonitor → Prometheus → Alertmanager.
4. Construire un dashboard Grafana qui mette en évidence la santé d'un service, et écrire les requêtes PromQL associées.
5. Migrer un Deployment vers une ressource Rollout d'Argo Rollouts, et choisir entre canary et blueGreen selon le contexte.

6. Écrire un `AnalysisTemplate` qui interroge Prometheus pendant la promotion, et lit automatiquement le résultat pour promouvoir ou rollback.
7. Configurer un *traffic routing* avancé (header-based, weighted) avec un contrôleur d'ingress.
8. Construire une chaîne d'alerting Alertmanager qui distingue ce qui réveille à 3h du matin de ce qui peut attendre lundi.
9. Caractériser ce que cette chaîne **ne fait pas** et nommer les briques complémentaires (chaos engineering, observabilité distribuée, RUM).

Modalités

- Rendu en binôme, sur un dépôt Git par binôme, avec un rapport `RAPPORT.md` à la racine.
- L'accompagnement formateur est oral et à la demande. Posez les questions à voix haute, montrez l'écran.
- Le squelette de départ vous est fourni dans le dossier `argocd/pulse-campus/` du dépôt de cours. Il prolonge le squelette du TP 2 : mêmes services, mêmes charts, **mais** les manifestes ont des marqueurs `# TODO` pour Rollout, `AnalysisTemplate`, `ServiceMonitor`, etc. **Forkez-le, ne le clonez pas.**
- Le rapport `RAPPORT.md` n'est pas un compte-rendu détaillé étape par étape — c'est une trace de votre raisonnement. On y attend des prises de position, des captures, des extraits de PromQL et de YAML, pas un copier-coller de la console.
- Aucune étape n'est chronométrée. Avancez à votre rythme, en validant chaque étape par sa section *Validation* avant de passer à la suivante.

Ce TP n'attend pas que vous codiez

Le code applicatif des trois services (annuaire, planning, notif) est **entièrement fourni** dans le squelette, *y compris l'instrumentation Prometheus* (endpoint `/metrics`, métriques RED, build info). Aucun écrit Node.js, Python ou Go ne vous est demandé.

Ce que vous écrivez, ligne par ligne, dans ce TP :

- des manifestes Kubernetes (Rollout, Service, Ingress, ServiceMonitor, PrometheusRule) ;
- des CRDs Argo Rollouts (`AnalysisTemplate`, `Experiment`) ;
- des manifestes ArgoCD (`Application`, `AppProject`) ;
- des fichiers `values.yaml` Helm pour configurer ce qui est templatisé ;
- des requêtes PromQL (à la main, dans Prometheus puis Grafana) ;
- des dashboards Grafana (à la souris, dans l'UI, puis exportés en JSON) ;
- du texte argumenté dans `RAPPORT.md`.

Lire le code des services est utile pour comprendre **ce que mesure** chaque label des métriques exposées. Mais vous n'aurez jamais à le modifier — sauf défi bonus explicite. Si vous voulez changer une convention de label, vous ouvrez une issue auprès des « équipes développement » (qui, en TP, est le formateur).

Stack 100 % CNCF (au statut près de Grafana)

Toutes les briques opérationnelles de ce TP sont des projets de la **Cloud Native Computing Foundation**, au statut *Graduated* (le plus haut, marqueur de maturité industrielle). C'est un choix délibéré : il vous permettra demain d'argumenter techniquement le coût et la pérennité de votre stack en entretien ou en comité d'archi.

Outil	Rôle dans le TP	Statut CNCF	Maturité
Kubernetes	runtime du cluster (via kind)	Graduated	depuis 2018
containerd	runtime conteneur sous kind	Graduated	depuis 2019
Helm	format de packaging des charts	Graduated	depuis 2020
ArgoCD (projet Argo)	GitOps, pull-based deployment	Graduated	depuis 2022
Argo Rollouts (projet Argo)	livraison progressive (canary, blueGreen)	Graduated	depuis 2022
Prometheus	métriques (collecte, stockage, alerting)	Graduated	depuis 2018
Alertmanager	routage et déduplication des alertes	Graduated (sous-projet Prometheus)	depuis 2018
OpenTelemetry (<i>extension</i>)	traces et métriques applicatives	Incubating	—
Jaeger (<i>extension</i>)	visualisation de traces	Graduated	depuis 2019
Kyverno (<i>extension</i>)	politiques as code	Graduated	depuis 2024
Cert-manager (<i>non utilisé dans le tronc</i>)	TLS automatique	Graduated	depuis 2024
Grafana	dashboards	Hors CNCF (Grafana Labs, AGPLv3)	—
Perses (<i>alternative à Grafana</i>)	dashboards	Sandbox (CNCF)	en émergence

Le cas Grafana. Grafana est le compagnon historique de Prometheus mais reste un projet de Grafana Labs (en dehors de la CNCF). C’est le seul outil non-CNCF de la stack. La CNCF a accueilli en 2023 **Perses**, un projet de dashboards positionné comme alternative open, en statut *Sandbox* — pas encore assez mûr pour un TP, mais à connaître. Vous noterez ce point d’écosystème explicitement dans votre synthèse finale, et vous saurez l’argumenter en entretien : « *si demain je veux une stack 100 % CNCF, je remplace Grafana par Perses ou par l’UI Prometheus + des recording rules pré-calculées ; en l’état je consens à un compromis pragmatique.* »

Pourquoi cela vous concerne. Hors CNCF, vous ne payez peut-être rien à l’achat, mais vous payez à long terme : licence qui change, gouvernance opaque, intégrations qui se ferment. Le marqueur *Graduated* CNCF n’est pas un sceau commercial — c’est un engagement de gouvernance ouverte, de neutralité de l’écosystème, et de garanties sur le rythme de release. C’est cet argument-là qui justifie, en réunion d’archi, le choix d’Argo Rollouts plutôt qu’une solution propriétaire incluse dans un PaaS.

Plateformes supportées

Identique au TP 2. Tout tourne sur un cluster kind local démarré par `make cluster-up`. Aucune VM, aucun cloud public.

Plateforme	Setup requis
macOS	Docker Desktop <i>ou</i> OrbStack. Commandes dans le terminal.
Windows 10/11	Docker Desktop en backend WSL2 + distro Ubuntu. Commandes dans WSL2.
Linux	Docker Engine.

Si vous arrivez de zéro et n'avez pas fait le TP 2, lisez d'abord la section *Plateformes supportées* du polycopié du TP 2 — elle reste valable mot pour mot.

Conventions du document

Objectif. ce qui doit être atteint à la fin de l'étape.

Contraintes. règles non-négociables (sécurité, naming, ressources).

Décomposition suggérée. un découpage en sous-tâches avec, pour chacune, les concepts à consulter dans la documentation officielle. Vous n'êtes pas obligé de la suivre — c'est une béquille pour ne pas rester bloqué.

Livrable. le fichier ou la sortie attendue dans votre dépôt.

Validation. ce que vous devez voir / vérifier (UI ArgoCD, UI Argo Rollouts, dashboards Grafana, requêtes PromQL) pour considérer l'étape réussie.

Pièges. erreurs classiques qui vous feront perdre du temps.

Défi bonus. extension qui pousse l'étape plus loin. Pour les binômes qui finissent vite ou veulent se challenger.

Comment lire ce poly

Mode	Pour qui	Ce que vous lisez
Brief de mission	Vous voulez vous débrouiller seul.	Objectif, Contraintes, Livrable, Validation.
Pas à pas	Vous voulez avancer sans vous perdre.	Tout, y compris la <i>Décomposition suggérée</i> et la doc officielle citée.
Défi	Vous avancez vite et voulez plus.	Idem brief + tous les <i>Défis bonus</i> et le <i>Boss final</i> en clôture.

Quel que soit le mode, lisez **avant tout la PARTIE I** (observabilité). Au TP 3, vous ne pouvez pas livrer progressivement ce que vous ne mesurez pas. Tenter la PARTIE II avant la PARTIE I est le moyen le plus court de perdre une demi-journée.

Pourquoi la livraison progressive, et pourquoi maintenant

Les limites du RollingUpdate natif

Kubernetes propose nativement, depuis ses débuts, une stratégie de déploiement appelée RollingUpdate. C'est ce que faisaient vos Deployment au TP 2. Le principe est simple : on remplace progressivement les anciens pods par les nouveaux, en respectant un maxSurge et un maxUnavailable.

C'est largement suffisant pour beaucoup de cas. C'est insuffisant dès que :

- l'erreur ne se voit pas sur le *health check* (le pod répond 200 OK mais sert une mauvaise réponse métier) ;
- l'erreur se voit après quelques minutes seulement (fuite mémoire, accumulation de connexions) ;
- l'erreur ne touche qu'un sous-ensemble du trafic (un type de requête, une région, un cohort) ;
- la conséquence d'un rollback tardif est coûteuse (perte de chiffre d'affaires, dégradation de l'expérience utilisateur sur des dizaines de minutes).

Dans tous ces cas, le RollingUpdate propage l'erreur à *toute* la flotte avant que vous ne réagissiez. Il n'a aucune notion de *bonne nouvelle* ou de *mauvaise nouvelle* applicative — il se contente de regarder les probes K8s.

Le principe : promouvoir sur preuve

La livraison progressive (*progressive delivery*) inverse la logique. Plutôt que de propager d'abord et de surveiller ensuite, on n'expose qu'une fraction du trafic, on mesure, et on ne propage que si les métriques restent acceptables.

Le mode *canary* — emprunté aux mineurs de charbon qui descendaient avec un canari pour détecter les gaz toxiques — consiste à envoyer un faible pourcentage du trafic (typiquement 5 %) vers la nouvelle version. On observe le taux d'erreur et la latence pendant quelques minutes. Si tout va bien, on monte à 25 %. Puis 50 %. Puis 100 %.

Le mode *blue/green* déploie l'intégralité de la nouvelle version en parallèle de l'ancienne sur une stack distincte. Une bascule unique du routage transfère tout le trafic d'un coup. Le rollback est instantané : on rebascule.

Le mode *shadow* (ou *mirror*) envoie les requêtes aux deux versions, mais ne renvoie au client que la réponse de l'ancienne. La nouvelle est testée sans risque pour l'utilisateur.

Pourquoi Argo Rollouts dans la suite d'ArgoCD

Argo Rollouts est un projet CNCF (statut *Graduated* depuis 2022, comme ArgoCD lui-même). Il introduit une CRD Rollout qui se substitue au Deployment natif et orchestre les stratégies ci-dessus. Trois raisons pour le choisir en suite logique du TP 2 :

1. **Cohérence d'outillage.** Même éditeur, mêmes conventions, même UI graphique. La courbe d'apprentissage par rapport au TP 2 est minimale.
2. **Intégration native avec ArgoCD.** Une Application ArgoCD peut suivre un Rollout exactement comme elle suit un Deployment. Le mode pull reste intact.
3. **AnalysisTemplate.** C'est le concept qui vous permet de brancher la décision de promotion sur n'importe quel *metric provider* (Prometheus, Datadog, Wavefront, New Relic, CloudWatch, web hook custom). C'est l'objet pédagogique central du TP.

L'alternative principale, **Flagger** (par FluxCD, également CNCF Graduated), fait la même chose avec une philosophie un peu différente — moins d'UI, plus d'automatisation par défaut. Vous le comparerez explicitement en étape 11.

Pourquoi Prometheus

Prometheus est le projet CNCF *Graduated* de référence pour les métriques en environnement Kubernetes. C'est le seul écosystème où le modèle *pull* (le serveur scrape les cibles, plutôt que les cibles qui poussent) s'aligne nativement avec le modèle de découverte de services de Kubernetes. Il offre :

- un format texte simple (`/metrics`) que tous les langages savent produire ;
- un langage de requête (PromQL) capable d'exprimer des SLI complexes en une ligne ;
- une intégration directe avec Argo Rollouts via les `AnalysisTemplate`.

Vous le déploierez via `kube-prometheus-stack`, qui empaquette Prometheus, Alertmanager, Grafana, Node Exporter et `kube-state-metrics` dans un seul chart Helm — l'industrie l'a adopté de facto. Grafana, bien qu'édité par Grafana Labs (en dehors de la CNCF), reste l'outil de visualisation standard pour Prometheus ; le TP l'utilise sans hésitation, en notant ce point d'écosystème dans la synthèse.

La promesse de cette chaîne

Quand vous aurez fini ce TP, un développeur qui pousse une PR sur main déclenchera la séquence suivante, sans qu'aucun humain n'intervienne :

1. La CI construit l'image et écrit un commit `image.tag: <sha>` dans le repo de configuration.
2. ArgoCD détecte le commit et synchronise le Rollout.
3. Argo Rollouts crée le nouveau `ReplicaSet`, route 5 % du trafic dessus.
4. Pendant 5 minutes, un `AnalysisTemplate` interroge Prometheus toutes les 30 secondes : taux d'erreur HTTP 5xx, latence p95.
5. Si les deux métriques restent sous leurs seuils, le rollout passe à 25 %, puis 50 %, puis 100 %.
6. Si l'une des deux dépasse, Argo Rollouts annule la promotion et bascule tout le trafic vers l'ancienne version. Le développeur reçoit une notification.

C'est la promesse. Vous allez la construire morceau par morceau.

Le même geste, trois paradigmes

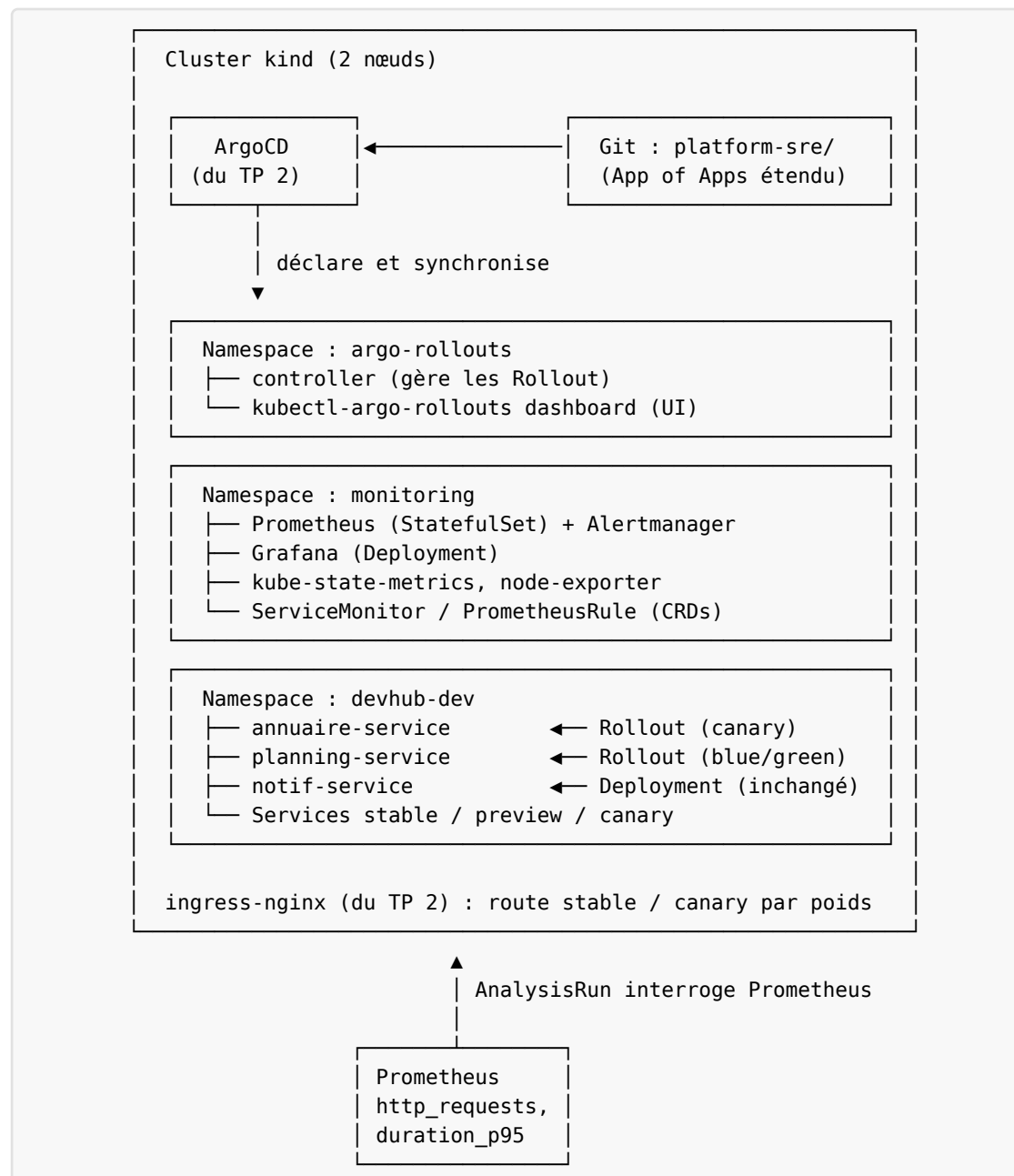
Ce tableau étend celui du TP 2. Vous y retrouverez la même colonne vertébrale, avec une troisième colonne pour le TP 3. Vous devrez le commenter dans votre synthèse finale.

Opération	TP 1 — K8s à la main	TP 2 — ArgoCD	TP 3 — ArgoCD + Argo Rollouts + Prometheus
Déployer une nouvelle version	<code>kubectl set image / re-apply</code>	<code>Commit image.tag</code> , ArgoCD sync	<code>Commit image.tag</code> , ArgoCD sync, Rollout promote sur preuve
Détecter qu'une nouvelle version dégrade le service	Quelqu'un appelle support	ArgoCD reste <i>Healthy</i> sans rien voir	AnalysisTemplate déclenche un rollback automatique
Faire un rollback	<code>kubectl rollout undo</code>	<code>git revert</code>	<code>kubectl argo rollouts undo</code> ou <code>auto</code>
Limiter l'impact d'une mauvaise version	Impossible — toute la flotte est touchée	Impossible — idem	5 % du trafic touché pendant 5 min max
Savoir si le service tient son SLO	Aucune métrique	Aucune métrique	Dashboard Grafana, alertes Alertmanager
Décider de promouvoir une release	Au feeling	Au feeling	Sur métriques mesurées
Justifier un déploiement à 17h vendredi	« ça devrait passer »	« ça devrait passer »	« le canary est positif sur 30 min »
Mesurer la fréquence de déploiement (DORA)	Logs CI	Logs CI ou Git	Rollout objects + Prometheus
Mesurer le <i>change failure rate</i> (DORA)	Souvenirs et tickets Jira	Idem	Rollouts annulés / promus
Tester une version sur un cohort d'utilisateurs	Impossible	Impossible	Header-based routing

L'écart entre les trois colonnes n'est pas un saut technique mais un saut *de maturité opérationnelle*. C'est l'objet du TP : passer de « *on déploie* » à « *on déploie en mesurant ce qu'on déploie* ».

Question à garder en tête tout le TP. Pour chaque ligne du tableau, qu'est-ce qui justifierait économiquement le passage de la colonne TP 2 à la colonne TP 3 ? Listez au moins deux opérations où la colonne TP 3 ne justifie pas son coût pour une petite équipe.

Architecture cible



Composant	Rôle	Outil	Vivant dans
argo-rollouts-controller	crée canary/stable RS, calcule les poids, lance les AnalysisRun	Argo Rollouts	namespace argo-rollouts
kube-prometheus-stack	scraping, stockage, alerting, dashboards	Prometheus Operator	namespace monitoring
ServiceMonitor	dit à Prometheus <i>qui</i> scraper	CRD	namespace du service
PrometheusRule	définit <i>alertes</i> et <i>enregistrements</i>	CRD	namespace du service ou monitoring
AnalysisTemplate	décrit <i>comment</i> mesurer pendant un canary	CRD	namespace du service
Rollout	remplace le Deployment, orchestre la stratégie	CRD	namespace applicatif
Service stable + canary	deux Services pointant sur le même Rollout via labels distincts	k8s natif	namespace applicatif

PARTIE I — Observabilité avant tout

Étape 0 — Outillage complémentaire

Objectif. disposer des CLI nécessaires pour piloter Argo Rollouts et inspecter Prometheus depuis le terminal. Le cluster du TP 2 est réutilisé tel quel.

Contraintes.

- vous installez les outils vous-mêmes ;
- vous documentez la version exacte de chaque outil dans `RAPPORT.md` ;
- aucun outil n'est installé *dans* le cluster à cette étape — uniquement côté poste.

Liste des outils requis en plus de ceux du TP 2 :

Outil	Pourquoi	Source
kubectl-argo-rollouts ≥ 1.7	sous-commande kubectl argo rollouts (status, pause, promote, abort, dashboard)	argoproj.github.io/argo-rollouts
promtool ≥ 2.50	valide les PrometheusRule (alertes et recording rules) hors cluster	prometheus.io
amtool (optionnel) ≥ 0.27	teste les routes Alertmanager hors cluster	prometheus.io
jq ≥ 1.7	parse les sorties JSON de l'API Prometheus	jqlang.github.io/jq

Décomposition suggérée.

1. Installez `kubectl-argo-rollouts` comme plugin `kubectl` (voir doc officielle). Vérifiez : `kubectl argo rollouts version` doit répondre.
2. Installez `promtool` (livré avec le tarball Prometheus). Vérifiez : `promtool --version`.
3. Si vous repartez d'un cluster vierge, relancez `make cluster-up` du TP 2. Le cluster reste utilisable tel quel — vous lui ajouterez des composants par ArgoCD.
4. Vérifiez que votre ArgoCD du TP 2 répond toujours sur `argocd.devhub.local`. Si non, refaites l'étape 5 du TP 2.

Livable. section *Outillage TP 3* du `RAPPORT.md` avec la sortie de `kubectl argo rollouts version` et `promtool --version`.

Pièges.

- `kubectl-argo-rollouts` est un plugin, pas une CLI standalone. Il s'appelle `kubectl argo rollouts ...`, pas `argo-rollouts ...`.
- Si vous avez à la fois `argocd` et `argo` (Workflows) dans le `PATH`, prenez soin de ne pas les confondre — leurs sous-commandes ont des noms proches.

Défi bonus. écrivez un `make tools-check-tp3` qui vérifie la présence des outils du TP 2 et du TP 3, avec une version minimale par outil. Sortie en erreur explicite si un outil manque.

Étape 1 — SLI, SLO, error budget : le vocabulaire avant le clavier

Objectif. être capable d'écrire un SLO réaliste pour un service web et de défendre ce choix. Cette étape est purement conceptuelle. Elle conditionne toutes les suivantes — vous ne mesurez bien que ce que vous savez nommer.

Décomposition suggérée.

1. Lisez le chapitre *Service Level Objectives* du livre *Site Reliability Engineering* de Google (disponible en ligne, gratuit). Notez les définitions exactes de SLI, SLO, SLA, *error budget*.
2. Lisez la page *RED method* de Tom Wilkie et la page *USE method* de Brendan Gregg. Sachez les distinguer (RED côté requêtes, USE côté ressources).
3. Pour chacun des trois services du TP 2, proposez **trois SLI candidates** et un **SLO par SLI**. Justifiez chaque seuil.
4. Pour chaque SLO, calculez l'*error budget* mensuel correspondant en minutes.

Livable. dans `RAPPORT.md`, un tableau par service avec les colonnes ci-dessous, plus, **séparément**, le PromQL pseudo-code de chaque SLI sous forme de bloc de code (pour ne pas surcharger le tableau).

Service	SLI	SLO	Error budget mensuel
annuaire	Disponibilité	99,5 %	3 h 36
annuaire	Latence p95	< 300 ms	3 h 36 / mois sous le seuil
annuaire	Fraîcheur des données	...	...
planning	...	...	...
notif	...	...	...

Exemple de PromQL pseudo-code pour les deux premières lignes de annuaire, à donner dans le rapport en dessous du tableau :

```
# Disponibilité (annuaire)
sum(rate(http_requests_total{service="annuaire", status_class!~"5.."}[5m]))
/
sum(rate(http_requests_total{service="annuaire"}[5m]))

# Latence p95 (annuaire)
histogram_quantile(
  0.95,
  sum(rate(http_request_duration_seconds_bucket{service="annuaire"}[5m])) by
(1e)
)
```

Validation. restituez à voix haute au formateur : « *pour annuaire, l'erreur budget est de X minutes par mois. Si on l'épuise en deux semaines, voici ce que je fais : ...* ». Si vous n'avez pas de réponse à la dernière partie, retournez lire.

Pièges.

- confondre SLI (le signal) et SLO (le seuil sur ce signal). Une *latence* n'est pas un SLI — « *la latence p95 sur 5 min* » est un SLI.
- choisir 99,99 % par défaut. Un SLO trop strict que vous ne tenez jamais est inutile (votre error budget est toujours négatif). Préférez 99,5 % tenu à 99,99 % rêvé.
- définir un SLO sans définir la *fenêtre* de calcul. « *99,5 % de disponibilité* » sur 1 minute ou sur 30 jours n'a pas la même signification.

Défi bonus. lisez le chapitre *Implementing SLOs* du livre *The Site Reliability Workbook* (Google). Distinguez SLO *event-based* et SLO *time-based*. Pour annuaire, lequel des deux modèles est le plus adapté ? Pourquoi ?

Étape 2 — Lire, comprendre et configurer l'instrumentation Prometheus

Objectif. comprendre *ce que mesure* un service déjà instrumenté, configurer les *buckets* d'histogramme et les *labels* via `values.yaml`, valider l'exposition `/metrics`. Aucune ligne de code à écrire — le squelette fourni a déjà tout codé dans les trois langages.

Choix du service. vous pouvez reprendre celui sur lequel vous avez travaillé au TP 2, ou en choisir un autre. La logique d'instrumentation est la même dans les trois langages — seule la bibliothèque change.

Ce qui est déjà fait pour vous (squelette).

- un endpoint HTTP `/metrics` exposé par chaque service au format texte Prometheus ;
- les métriques suivantes émises automatiquement par un *middleware* intégré au code :
 - `http_requests_total` (Counter, labels `method`, `route`, `status_class`) ;
 - `http_request_duration_seconds` (Histogram, mêmes labels, buckets paramétrables) ;
 - `<service>_build_info` (Gauge constante = 1, labels `version`, `commit`, `language`) ;
- utilisation des bibliothèques officielles CNCF Prometheus : `prom-client` (Node.js), `prometheus_client` (Python), `prometheus/client_golang` (Go) ;
- le label `status_class` est *grouped* ("`2xx`", "`3xx`", "`4xx`", "`5xx`") — pas d'explosion de cardinalité ;
- le label `route` utilise le *pattern* `(/users/:id)` — pas l'URL brute.

Ce que vous faites dans cette étape.

- vous **lisez** le code du middleware d'instrumentation du service que vous avez choisi (un fichier court, commenté, à la racine de `src/ / app/ / cmd/`) ;
- vous décidez des **buckets** de l'histogramme en fonction de vos SLOs de l'étape 1, et vous les configurez via `chart/values.yaml` (clé `metrics.buckets`) ;
- vous décidez si vous voulez activer la métrique optionnelle `business_event_total` (Counter applicatif paramétrable) ;
- vous validez l'exposition `/metrics` en local et formatez la sortie.

Décomposition suggérée.

1. Lisez la page *Naming* des conventions Prometheus et la page *Histograms and summaries*. Comprenez pourquoi un Histogram est presque toujours préférable à un Summary côté serveur.
2. Ouvrez le fichier du middleware (`src/metrics.js`, `app/metrics.py`, ou `cmd/metrics.go` selon le service). Lisez-le. Identifiez : où est créé chaque Counter et Histogram, où sont définis les labels, comment les buckets sont injectés.
3. Réfléchissez aux buckets : si votre SLO est $p95 < 300\text{ ms}$, il faut un bucket à 300 ms (et idéalement aussi 100 ms, 200 ms, 500 ms, 1 s) pour que le quantile soit exact à ce niveau. Documentez vos buckets dans le rapport.
4. Modifiez `chart/values.yaml` : remplissez la liste `metrics.buckets` avec votre choix. Le chart la transmettra via une variable d'environnement à l'instrumentation.
5. Lancez l'image en local : `docker run -p 8080:8080 -e METRICS_BUCKETS="0.05,0.1,0.2,0.3,0.5,1,2" <image>`. Faites quelques requêtes : `curl localhost:8080/students` (ou `/slots`, `/events`).
6. `curl localhost:8080/metrics` — la sortie est du texte aligné avec des `#` `HELP` et `# TYPE`. Validez le format avec `promtool check metrics <(curl -s localhost:8080/metrics)>`.

Livrable.

- `chart/values.yaml` mis à jour avec `metrics.buckets` justifié dans le rapport ;
- une capture de la sortie `curl /metrics` montrant vos buckets effectifs ;
- dans le rapport, en deux à trois phrases : « *pour annuaire, mes buckets sont X, Y, Z parce que mon SLO p95 est de N ms.* »

Validation.

- `curl localhost:8080/metrics` retourne du Prometheus valide après vos changements de buckets ;
- `promtool check metrics ...` ne signale aucune erreur ;
- les buckets que vous voyez dans la sortie sont bien ceux que vous avez mis dans `values.yaml`, pas ceux par défaut.

Pièges.

- oublier que les buckets sont *cumulatifs* dans Prometheus — chaque ligne `le="0.3"` compte aussi tout ce qui est sous 0.1 et 0.2. C'est voulu, mais ça déroute au premier coup d'œil.
- choisir des buckets uniformes (0.1, 0.2, 0.3, ...) : presque toujours inutile. Préférez une progression logarithmique adaptée à votre SLO.
- confondre `status` et `status_class` : le code expose `status_class` (les 2xx, 3xx, etc.) pour éviter l'explosion de cardinalité. Vos requêtes PromQL utiliseront `status_class!` `~"5.."`, pas `status!~"5.."`.

Défi bonus.

- lisez l'option `business_event_total` dans le code du middleware. Configurez-la via `values.yaml` pour qu'elle compte un événement métier de votre choix (ex. *nombre d'étudiants listés*). Vérifiez qu'il apparaît dans `/metrics`.
- dans le code, identifiez précisément le mécanisme qui empêche l'explosion de cardinalité. À votre avis, où l'instrumentation pourrait-elle encore mal tourner si un développeur ajoute une route mal écrite ?

Étape 3 — Installer kube-prometheus-stack via ArgoCD

Avant (TP 2) / Maintenant (TP 3). Au TP 2, le seul opérateur installé à la main dans le cluster était ArgoCD lui-même, via `helm install`. Au TP 3, tout passe par ArgoCD : on déclare la stack d'observabilité comme une Application Helm, et ArgoCD se charge du déploiement. C'est le pattern *Argo gère Argo* élargi à *Argo gère sa stack d'observabilité*.

Objectif. déployer Prometheus, Alertmanager, Grafana, Node Exporter et kube-state-metrics, le tout en mode GitOps via une Application ArgoCD.

Contraintes.

- source : le chart Helm `prometheus-community/kube-prometheus-stack` ;
- destination : namespace `monitoring` (à créer via `syncOptions: CreateNamespace=true`) ;
- une Application enfant du *App of Apps* du TP 2, vivant dans `platform-sre/apps/observability/kube-prometheus-stack.yaml` ;
- le mot de passe admin de Grafana initial est défini dans un Secret créé hors-Git, ou via *Sealed Secrets* (cf. défi bonus) ;
- Grafana est exposé sur `grafana.devhub.local` via l'ingress (réutilisez l'ingress-nginx du TP 2) ;
- Prometheus est exposé sur `prometheus.devhub.local` (UI en lecture seule) ;
- le chart est figé à une version précise (`targetRevision`) — pas de HEAD mouvant.

Décomposition suggérée.

1. Lisez la doc du chart `kube-prometheus-stack`. La liste des `values.yaml` est longue. Identifiez quels sous-charts sont activés par défaut (Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics) et lesquels vous voulez désactiver pour ne pas saturer votre kind.
2. Sur un cluster kind 2 nœuds, désactivez le sous-chart `prometheusOperator.admissionWebhooks.patch` s'il vous pose problème. Désactivez aussi les *probes* de la control plane (`kubeControllerManager`, `kubeScheduler`, `kubeEtcd`) — kind ne les expose pas.
3. Écrivez l'Application ArgoCD avec une source de type Helm (référence au chart distant + `values`). Préférez `valuesFrom` un fichier dans votre repo `platform-sre/` plutôt qu'inliné.
4. Commitez. ArgoCD doit créer l'Application, puis tous les pods. Patience : la première install pèse plusieurs Go.

Livrable.

- le manifeste `platform-sre/apps/observability/kube-prometheus-stack.yaml` ;
- le fichier `values.yaml` du chart ;
- capture d'écran de l'UI ArgoCD montrant l'Application *Synced + Healthy* avec tous ses pods *Running*.

Validation.

- `kubectl get pods -n monitoring` montre Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics tous *Running* ;
- l'UI Prometheus répond sur `prometheus.devhub.local` ;
- l'UI Grafana répond sur `grafana.devhub.local` (admin / mot de passe défini) ;
- dans Prometheus, l'onglet *Status* → *Targets* montre des cibles *UP* (kube-state-metrics, node-exporter, l'API K8s).

Pièges.

- le chart kube-prometheus-stack installe **beaucoup** de CRDs. Le passage Argo en OutOfSync après le premier sync est souvent dû à un mismatch de apiVersion sur un de ces CRDs — lisez les notes de version du chart.
- sur kind, certains sous-charts (control-plane scraping) ne marcheront pas. Désactivez-les explicitement, n'essayez pas de les forcer.
- laissez Grafana en mot de passe par défaut prom-operator : à la moindre exposition publique, c'est l'incident. Forcez une valeur, même dummy.

Défi bonus.

- installez en plus le chart sealed-secrets (CNCF *sandbox*). Chiffrez le mot de passe Grafana en SealedSecret committable. Documentez le workflow de rotation.
- étudiez la possibilité de scraper Prometheus *par Prometheus lui-même* (méta-observabilité). Quels signaux sont utiles ? Comment alerter sur un Prometheus qui ne scrape plus ?

Étape 4 — Brancher votre service : ServiceMonitor + premier dashboard

Objectif. que Prometheus scrape effectivement votre service et que vous voyiez ses métriques dans un dashboard Grafana minimal mais lisible.

Contraintes.

- le scraping passe par un ServiceMonitor (CRD posée par le Prometheus Operator), pas par une annotation prometheus.io/scrape sur le Service ;
- le ServiceMonitor vit dans le namespace du service (devhub-dev), pas dans monitoring ;
- le ServiceMonitor est versionné dans le chart Helm de votre service (template conditionnel monitoring.enabled) — vous l'activez par values.yaml par environnement ;
- le dashboard Grafana est *exporté en JSON* et commité dans platform-sre/dashboards/<service>.json. Il doit comporter au minimum quatre panneaux : *Request rate (RPS)*, *Error rate*, *Latency p50/p95/p99*, *Build info* (le tag d'image actif).
- **toutes les requêtes PromQL du dashboard sont documentées dans le rapport.**

Décomposition suggérée.

1. Étudiez la doc du Prometheus Operator sur les ServiceMonitor. Comprenez le mécanisme de sélection (selector + label sur le Service).
2. Ajoutez le label release: <nom-de-la-release-Prometheus> sur votre Service. Sans ce label, le Prometheus du chart ne le verra pas.
3. Templatisiez le ServiceMonitor dans chart/templates/servicemonitor.yaml du service. Activez-le dans values-dev.yaml.
4. Committez. ArgoCD synchronise le chart, le ServiceMonitor apparaît, Prometheus le détecte (vérifiez dans *Status* → *Targets*).
5. Dans Grafana, créez un nouveau dashboard. Ajoutez les quatre panneaux. Utilisez les variables \$service, \$namespace, \$instance pour rendre le dashboard réutilisable.

6. Exportez le dashboard en JSON, commitez-le.

Livrable.

- le template `servicemonitor.yaml` ajouté au chart Helm du service ;
- le dashboard Grafana JSON dans `platform-sre/dashboards/` ;
- dans le rapport, les quatre requêtes PromQL des panneaux avec une phrase d'explication chacune.

Validation.

- dans Prometheus → *Status* → *Targets*, votre service apparaît avec `UP=1` ;
- une requête `rate(http_requests_total[5m])` retourne des valeurs non nulles ;
- dans Grafana, votre dashboard affiche du trafic après quelques `curl` envoyés à votre service ;
- vous pouvez expliquer ce que chaque panneau mesure et quel SLO il sert.

Pièges.

- oublier le label `release: <release>` sur le Service → Prometheus ne le voit pas, *aucune erreur visible*. C'est le piège le plus fréquent.
- écrire une requête `histogram_quantile(0.95, http_request_duration_seconds_bucket)` sans agrégation : vous obtenez un quantile par instance, illisible. Toujours `sum(rate(...)) by (le)` d'abord.
- confondre `rate()` et `irate()` : `irate` réagit aux pics instantanés, `rate` lisse sur la fenêtre. Pour des dashboards, presque toujours `rate`.

Défi bonus.

- écrivez une PrometheusRule qui contient les *alertes* du service (taux d'erreur > 1 %, latence p95 > 300 ms, plus de mises à jour depuis 24h). Templatisez-la dans le chart Helm. Validez-la avec `promtool check rules`.
- écrivez un panneau Grafana « *error budget consommé* » qui affiche, en pourcentage, l'érosion du budget d'erreur sur les 30 derniers jours. La formule est non triviale — c'est l'intérêt.

PARTIE II — Livraison progressive

Étape 5 — Du Deployment au Rollout

Avant (TP 2) / Maintenant (TP 3). Au TP 2, votre chart Helm produisait un Deployment. ArgoCD synchronisait, K8s faisait un RollingUpdate, et la nouvelle version remplaçait l'ancienne en quelques secondes — sans aucune mesure de l'impact applicatif. Au TP 3, vous remplacez le Deployment par un Rollout. La synchronisation Git → cluster reste la même, mais la *propagation interne au cluster* devient orchestrée et observable.

Objectif. installer le contrôleur Argo Rollouts via ArgoCD, puis migrer le Deployment de votre service vers un Rollout en mode canary minimaliste (étapes simples, sans Analysis).

Contraintes.

- Argo Rollouts est installé via une Application ArgoCD (chart argo/argo-rollouts), dans le namespace argo-rollouts ;
- le *dashboard* d'Argo Rollouts est activé (dashboard.enabled: true) et exposé via ingress sur rollouts.devhub.local ;
- vous remplacez le Deployment du service par un Rollout dans chart/templates/rollout.yaml. Le Deployment est **supprimé**, pas conservé en doublon ;
- le Rollout référence deux Services :
 - un Service *stable* (le trafic normal) ;
 - un Service *canary* (le trafic de canari) ;
 - les deux Services pointent vers le même Rollout mais via des sélecteurs distincts générés automatiquement par Argo Rollouts ;
- la stratégie canary initiale comporte **trois étapes** simples : setWeight: 20, pause: {duration: 30s}, setWeight: 50, pause: {duration: 30s}, setWeight: 100. Pas encore d'Analysis — c'est l'objet de l'étape 7.
- l'ingress route vers le Service stable (Argo Rollouts gère le split avec ingress-nginx — lisez la doc *Traffic Management*).

Décomposition suggérée.

1. Lisez la page *Architecture* d'Argo Rollouts. Comprenez la différence entre Rollout, ReplicaSet, Services stable/canary, et le rôle du contrôleur.
2. Écrivez l'Application ArgoCD qui installe Argo Rollouts. Synchronisez. Vérifiez que les CRDs sont posées (kubectl get crd | grep rollouts).
3. Dans le chart Helm de votre service, dupliquez deployment.yaml en rollout.yaml. Adaptez : apiVersion: argoproj.io/v1alpha1, kind: Rollout, ajoutez strategy.canary avec les steps simples. **Supprimez deployment.yaml.**
4. Ajoutez deux Services dans le chart : service-stable.yaml et service-canary.yaml. Lisez la doc *Traffic Management with NGINX Ingress* pour les conventions de labels.
5. Commitez. ArgoCD synchronise. Le Rollout se crée, le contrôleur crée un seul ReplicaSet (pas de canari à la première sync).
6. Poussez un commit qui change l'image (par exemple, ajoutez une variable d'environnement triviale). Observez en parallèle : kubectl argo rollouts get rollout <nom> --watch et le dashboard Argo Rollouts.

Livrable.

- le manifeste Application qui installe Argo Rollouts ;
- le chart Helm modifié (rollout + 2 services) ;
- une capture de kubectl argo rollouts get rollout <nom> pendant un canary en cours, montrant le poids actuel et l'étape.

Validation.

- le dashboard Argo Rollouts (`rollouts.devhub.local`) affiche votre Rollout ;
- un changement de tag d'image dans Git déclenche un canary qui passe successivement par 20 %, 50 %, 100 % avec les pauses ;
- pendant la phase 20 %, environ 20 % des requêtes envoyées sur l'ingress du service touchent la nouvelle version (vérifiable par les logs ou par le label version que vous avez exposé en étape 2).

Pièges.

- oublier de supprimer `deployment.yaml` après avoir créé `rollout.yaml` : ArgoCD croit que les deux doivent coexister, et vous vous retrouvez avec deux ReplicaSets qui se battent pour les pods. Le Deployment doit *disparaître du chart*.
- se tromper sur le trafic routing : si vous n'utilisez pas l'ingress controller comme *router*, Argo Rollouts ne peut faire que du *traffic split* basé sur le nombre de réplicas (approximatif). C'est utile à savoir mais pas suffisant à 5 % de précision.
- oublier d'ajouter les annotations spécifiques à `ingress-nginx` (`nginx.ingress.kubernetes.io/canary*`) sur l'Ingress canary.

Défi bonus.

- inspectez les ressources que crée Argo Rollouts pendant un canary : `kubectl get rs,ingress -n devhub-dev -l app=<service>`. Que voyez-vous d'inattendu ? À quoi sert l'Ingress *secondaire* créé par le contrôleur ?

Étape 6 — Canary manuel : pause, promote, abort

Objectif. prendre la main sur le déroulé d'un canary depuis le terminal et le dashboard. Comprendre les transitions d'états du Rollout.

Contraintes.

- vous transformez vos steps en une vraie séquence avec un *pause manuel* : `setWeight: 10, pause: {}` (durée indéfinie), `setWeight: 50, pause: {duration: 1m}`, `setWeight: 100` ;
- vous testerez **trois scénarios** distincts et les documenterez chacun :
 1. Promotion normale (canary OK → vous cliquez/tapez *promote*).
 2. Annulation explicite (canary OK mais vous décidez d'abort → bascule vers stable).
 3. Promotion forcée sans inspection (`promote --full`) pour comprendre le risque que cela couvre.

Décomposition suggérée.

1. Lisez les commandes `kubectl argo rollouts get, pause, promote, abort, set image`. Faites-les sortir une fois en `--help` pour mémoriser.

2. Modifiez les steps du Rollout. Commitez. ArgoCD synchronise.
3. Déclenchez un nouveau canary (changez l'image tag par un commit). Observez : le Rollout passe à 10 %, puis se met en Paused.
4. **Scénario 1.** Tapez `kubectl argo rollouts promote <nom>`. Le canary passe à 50 %, attend 1 min, passe à 100 %.
5. **Scénario 2.** Relancez un canary, attendez la pause à 10 %, tapez `kubectl argo rollouts abort <nom>`. Observez : le poids canari descend à 0, l'ancien ReplicaSet reprend tout. Faites un `git revert` pour ramener Git en cohérence.
6. **Scénario 3.** Relancez un canary, tapez `kubectl argo rollouts promote <nom> --full`. Le canary saute toutes les étapes restantes. Pourquoi est-ce dangereux ? Quand serait-ce justifié (incident, vraie urgence) ?

Livrable.

- section *Pilotage canary manuel* du rapport avec les trois scénarios, chacun documenté par : commande exacte, capture du dashboard avant/après, observation.
- une réponse argumentée à : « *dans quels cas un promote --full est-il acceptable en production ? Quelles précautions prendre ?* »

Validation.

- vous pilotez les transitions sans hésiter, depuis le terminal et le dashboard, en moins de cinq commandes pour chaque scénario.

Pièges.

- abort ramène le poids à 0 dans le cluster, mais **ne modifie pas Git**. Vous vous retrouvez en *drift* (le cluster utilise l'ancienne image, Git pointe sur la nouvelle). Vous devez décider : `git revert` pour aligner Git sur le cluster, ou `promote` pour aligner le cluster sur Git.
- oublier qu'abort ne nettoie pas l'historique du Rollout — un `get rollout` montre toujours l'attempt avorté. C'est voulu (audit).

Défi bonus.

- configurez un *experiment* Argo Rollouts (autre CRD que Rollout) qui teste deux versions en parallèle sur 1 % du trafic chacune, sans intention de promouvoir. À quel usage cela servirait-il ?

Étape 7 — AnalysisTemplate : la promotion sur preuve

Objectif. brancher la décision de promotion sur Prometheus. Aucune intervention humaine n'est plus nécessaire pendant un canary réussi.

Contraintes.

- vous écrivez un `AnalysisTemplate` dans le chart Helm du service (`templates/analysis-template.yaml`) ;
- l'analyse interroge Prometheus à `http://prometheus-operated.monitoring.svc.cluster.local:9090` (le Service interne posé par le Prometheus Operator) ;
- **deux métriques** au minimum :
 1. *Taux d'erreur* — la proportion de 5xx sur le total de requêtes vers la version *canary* doit rester sous 1 % ;
 2. *Latence p95* — la latence p95 de la version *canary* doit rester sous votre SLO (étape 1).
- chaque métrique est échantillonnée toutes les 30 secondes pendant 5 minutes (10 mesures) ;
- une métrique en *Failed* déclenche immédiatement un rollback ;
- le Rollout référence l'`AnalysisTemplate` dans un step analysis placé entre le `setWeight: 25` et le `setWeight: 50` ;
- vous **devez** différencier la version canary de la version stable dans les requêtes PromQL. Indice : utilisez le label `rollouts-pod-template-hash` posé par Argo Rollouts, ou le label version que vous avez injecté à l'étape 2.

Décomposition suggérée.

1. Lisez la doc *Analysis* d'Argo Rollouts. Étudiez le format `AnalysisTemplate` et la sémantique de `failureLimit`, `inconclusiveLimit`, `successCondition`.
2. Écrivez d'abord les requêtes PromQL dans l'UI Prometheus, à la main. Vérifiez qu'elles retournent les valeurs attendues sur du trafic réel (envoyé via curl en boucle).
3. Templatisiez l'`AnalysisTemplate`. Référez-le dans les steps du Rollout.
4. Commitez. Déclenchez un canary. Observez dans le dashboard : un nouvel objet `AnalysisRun` apparaît pendant l'étape concernée. Cliquez dessus pour voir les mesures successives.
5. **Provoquez un échec.** Modifiez le code du service pour renvoyer du 500 sur 5 % des requêtes (sous un flag d'environnement). Déclenchez un canary. L'`AnalysisRun` doit basculer en *Failed*, le Rollout en *Degraded*, le trafic redescend à 0 sur le canary.

Livrable.

- le `AnalysisTemplate` commité dans le chart ;
- capture de l'`AnalysisRun` réussi (canary OK, promotion auto) ;
- capture de l'`AnalysisRun` échoué (rollback auto) ;
- dans le rapport, une discussion sur le choix des seuils et de la durée.

Validation.

- vous démontrez en direct au formateur un canary qui se promet tout seul (cas nominal) ;
- vous démontrez en direct un canary qui se rollback tout seul (cas dégradé, version qui renvoie du 500) ;
- vous expliquez ce que dit `kubectl argo rollouts get analysisrun <nom>` à chaque mesure.

Pièges.

- mettre des seuils trop laxes (5 % d'erreur acceptable) : vous laissez passer des régressions évidentes. Trop stricts (0,1 %) : un pic statistique sur 10 mesures fait échouer un rollout valide.
- oublier de filtrer la requête PromQL sur le pod-template-hash du canary : vous mesurez la moyenne canary + stable, ce qui dilue le signal.
- laisser la durée d'analyse trop courte (1 min) : sur du trafic faible (TP en local), le quantile p95 manque de points. Préférez 3 à 5 minutes.

Défi bonus.

- ajoutez une troisième métrique : *taux de saturation CPU du canary*. Utilisez `container_cpu_usage_seconds_total` (exporté par cAdvisor / kubelet).
- construisez un AnalysisTemplate *réutilisable* (avec des paramètres `service`, `namespace`, `errorThreshold`). Référencez-le depuis plusieurs services avec des seuils différents.

Étape 8 — Blue/Green : autre stratégie, autre arbitrage

Objectif. déployer un second service en mode blueGreen, comprendre ce que cette stratégie change pour le routage, le rollback, et le coût en ressources.

Contraintes.

- vous appliquez le mode blueGreen au service planning (ou un autre que celui de l'étape 7) ;
- les deux versions tournent **simultanément** à pleine capacité ;
- la bascule (`activeService` → nouvelle version) est manuelle pour la première démonstration, puis automatisée avec un `prePromotionAnalysis` à la seconde ;
- l'ancienne version est conservée pendant `scaleDownDelaySeconds: 300` (5 min) après bascule, le temps de pouvoir rollback instantanément.

Décomposition suggérée.

1. Étudiez la section *BlueGreen Deployment* de la doc Argo Rollouts. Notez les champs `activeService`, `previewService`, `autoPromotionEnabled`.
2. Templatisiez le Rollout avec `strategy.blueGreen`. Ajoutez le `previewService` (Service K8s distinct pour tester la nouvelle version avant bascule).
3. Exposez le `previewService` via un ingress séparé (`planning-preview.devhub.local`). Lui n'est joignable que par l'équipe interne.
4. Déclenchez un déploiement. Observez : la nouvelle version se déploie à 100 %, mais le `previewService` est seul à pointer dessus. Le trafic utilisateur passe encore sur l'ancienne via `activeService`.
5. Validez manuellement la preview (`curl planning-preview.devhub.local`). Tapez `kubectl argo rollouts promote <nom>` : le `activeService` bascule.

6. Ajoutez un `prePromotionAnalysis` qui vérifie la preview pendant 2 minutes avant d'autoriser la bascule.

Livrable.

- le chart Helm du second service migré en blueGreen ;
- un schéma comparatif canary vs blueGreen dans le rapport (avantages, inconvénients, cas d'usage) ;
- capture d'une bascule manuelle réussie.

Validation.

- pendant le déploiement, `kubectl get pods -n devhub-dev -l app=planning` montre **deux fois** plus de pods que d'habitude ;
- le `previewService` répond la nouvelle version, le `activeService` répond encore l'ancienne ;
- après `promote`, le `activeService` répond la nouvelle version, l'ancien `ReplicaSet` reste actif 5 min puis disparaît.

Pièges.

- oublier que blueGreen *double* la consommation de ressources pendant la bascule. Sur `kind`, surveillez `kubectl top nodes` — vous pouvez saturer.
- confondre `previewService` (Service spécifique blueGreen) et l'ingress *canary* (spécifique canary). Les deux ressources et les deux stratégies sont indépendantes — n'essayez pas d'en mixer.

Défi bonus.

- quand utiliseriez-vous blueGreen plutôt que canary ? Donnez deux exemples concrets de votre choix, et défendez-les en deux phrases chacun.
- pouvez-vous brancher un test E2E (un script qui appelle 50 endpoints du service en assertion) comme `prePromotionAnalysis` Web Hook ? Esquissez l'architecture.

Étape 9 — Routage avancé : header-based pour les tests internes

Objectif. envoyer une fraction du trafic vers la nouvelle version non pas au hasard, mais sur un critère métier — par exemple un header HTTP `X-Beta-User: true`.

Contraintes.

- le routage par header est géré par ingress-nginx via les annotations `nginx.ingress.kubernetes.io/canary-by-header` ;

- vous configurez votre Rollout (ou directement l’Ingress du canary) pour qu’une nouvelle version reçoive **seulement** les requêtes portant X-Beta-User: true, indépendamment du setWeight ;
- vous documentez l’usage métier : « *cela permettrait à l’équipe produit de tester chaque release sur leurs propres comptes avant n’importe quel utilisateur.* »

Décomposition suggérée.

1. Lisez la documentation *Canary* d’ingress-nginx, section *Header-based*. Notez les trois annotations clés : canary, canary-by-header, canary-by-header-value.
2. Modifiez la définition de l’Ingress canary (générée par Argo Rollouts ou définie dans votre chart) pour ajouter ces annotations.
3. Démontrez : curl <service>.devhub.local répond la version stable ; curl -H "X-Beta-User: true" <service>.devhub.local répond la nouvelle version.

Livrable.

- le chart ou la configuration du Rollout / Ingress mis à jour ;
- dans le rapport, une démonstration en curl des deux comportements.

Validation.

- sans header, le service répond toujours la version stable ;
- avec le header, le service répond la nouvelle version ;
- vous expliquez en deux phrases comment cette technique se combinerait avec AnalysisTemplate dans un workflow d’équipe produit.

Pièges.

- oublier qu’avec canary-by-header, le canary-weight est *ignoré* pour les requêtes qui matchent le header. Ce n’est pas une combinaison additive — le header **gagne**.

Défi bonus.

- utilisez canary-by-cookie à la place de l’en-tête : le client est routé vers la nouvelle version *de manière persistante* via un cookie. Pourquoi est-ce mieux pour une véritable beta utilisateurs ?

PARTIE III — Synthèse et maîtrise

Étape 10 — Alerting Alertmanager et notifications Rollouts

Objectif. faire en sorte que les humains soient prévenus *au bon moment, sur le bon canal*, ni avant ni après.

Contraintes.

- vous écrivez deux PrometheusRule au moins :
 1. « *Error rate au-delà de 1 % sur 5 min* » — sévérité page (réveille).
 2. « *Latence p95 dégradée sur 30 min* » — sévérité ticket (attend lundi).
- Alertmanager est configuré pour router :
 - les alertes severity: page vers un webhook *primaire* (mockez avec `webhook.site`) ;
 - les alertes severity: ticket vers un webhook *secondaire* différent ;
- Argo Rollouts est configuré pour notifier (via `argo-rollouts/notifications`) sur chaque rollout terminé : succès vers un troisième webhook, échec vers le premier.
- les notifications contiennent au minimum : nom du Rollout, version sortante, version entrante, durée totale, conclusion (Promoted/Aborted).

Décomposition suggérée.

1. Lisez les pages *Routing* et *Receivers* d'Alertmanager. Comprenez `route`, `group_by`, `group_wait`, `repeat_interval`.
2. Écrivez vos deux PrometheusRule. Validez localement avec `promtool check rules`.
3. Configurez Alertmanager via le sous-chart `alertmanager` de `kube-prometheus-stack` (section `alertmanager.config`).
4. Mettez en place les notifications Argo Rollouts : suivez la doc *Notifications*. Le service `notifications-controller` lit un ConfigMap `argo-rollouts-notification-configmap` et un Secret pour les credentials.
5. Provoquez les trois cas (page, ticket, rollout terminé) un par un.

Livrable.

- vos PrometheusRules dans le chart ;
- la configuration Alertmanager dans `platform-sre/` ;
- la configuration des notifications Rollouts ;
- captures des trois webhooks recevant leurs payloads.

Validation.

- une charge artificielle qui provoque 5 % de 5xx déclenche au bout de 5 minutes une alerte sur le webhook *primaire* ;
- une charge qui ralentit modérément le service déclenche au bout de 30 min une alerte sur le webhook *secondaire* ;
- chaque rollout (Promoted ou Aborted) génère une notification sur le webhook correspondant.

Pièges.

- mettre toutes les alertes en severity: `critical` : à 3h du matin, tout sonne, plus personne ne lit. La gradation page / ticket est ce qui rend l'oncall vivable.

- oublier `repeat_interval` : une alerte qui reste *firing* est resignalée toutes les 4h par défaut. Mauvais réglage → spam.
- écrire une règle d'alerte sans `for`: X_m : la moindre fluctuation transitoire de PromQL fait trigger. Toujours stabiliser avec une fenêtre `for`.

Défi bonus.

- intégrez Alertmanager avec un vrai service (Slack, Discord, Telegram bot, etc.). Pas de webhook mocké. Documentez la rotation du token.
- écrivez un *runbook* pour chaque alerte : un fichier markdown lié depuis l'annotation `runbook_url` de la `PrometheusRule`. Que doit-il contenir au minimum pour être utile à 3h du matin ?

Étape 11 — Comparer Argo Rollouts, Flagger, et la rolling-update native

Objectif. prendre du recul. Aucun outil n'est universellement supérieur. Vous devez savoir défendre votre choix.

Livrable. dans `RAPPORT.md`, la matrice ci-dessous **complétée et argumentée** (chaque cellule = une note de 0 à 5 + une phrase de justification) :

Critère	RollingUpdate natif	Argo Rollouts	Flagger
Courbe d'apprentissage	...	...	...
Intégration avec ArgoCD (workflow GitOps)	...	...	...
Intégration avec Flux (workflow GitOps)	...	...	...
Variété des stratégies (canary, blueGreen, A/B, shadow)	...	...	...
Variété des metric providers	...	...	...
UI / dashboard prêt à l'emploi	...	...	...
Coût opérationnel dans le cluster	...	...	...
Adapté à un mesh (Linkerd, Istio)	...	...	...
Communauté / fréquence des releases	...	...	...
Risque si le contrôleur tombe en plein canary	...	...	...

Validation. restituez la matrice au formateur en cinq minutes maximum, sans la relire. Argumentez chaque cellule où votre note diffère de 3.

Étape 12 — Synthèse obligatoire : « ma chaîne de release est-elle production-ready ? »

Objectif. l'étape de synthèse. C'est celle qui distingue « *j'ai cliqué sur promote* » de « *je comprends ce qu'est une chaîne de release sûre* ».

Livable 1 — Rétrospective TP 2 → TP 3. reprenez le tableau du chapitre « *Le même geste, trois paradigmes* ». Pour chaque ligne, commentez :

- en une phrase, qu'est-ce que la colonne TP 3 vous a fait *vraiment ressentir* (plus rapide ? plus contraint ? plus rassurant ?) ;
- identifiez **au moins deux opérations** pour lesquelles le surcoût opérationnel du TP 3 n'est *pas* justifié si vous étiez dans une petite startup de 3 personnes ;
- identifiez **l'opération** qui, à elle seule, justifierait à vos yeux le passage du TP 2 au TP 3 si vous étiez responsable plateforme dans une PME en croissance.

Livable 2 — Ce que cette chaîne ne sait toujours pas faire. un chapitre structuré ainsi. Pour chacun des sept thèmes ci-dessous, expliquez en 5 à 10 lignes :

1. **Le risque concret** que vous prenez si vous déployez DevHub Campus SRE en l'état chez un vrai client.
2. **L'outil complémentaire** que vous installeriez en plus pour traiter le risque.
3. **Une référence** (doc officielle, article, vidéo) que vous garderiez sous la main.

Thème	Mot-clé à creuser
Traçabilité distribuée (un appel utilisateur → tous les services traversés)	OpenTelemetry, Jaeger, Tempo
Logs centralisés et corrélés aux métriques	Loki, Fluent Bit, exemplars
Mesure côté utilisateur (latence réelle navigateur)	RUM, Web Vitals
Chaos engineering applicatif	Chaos Mesh, LitmusChaos, Pumba
Politique d'admission des manifests	Kyverno, OPA Gatekeeper
Signature des images et provenance (chaîne d'approvisionnement)	Sigstore (cosign), in-toto, SLSA
Backup applicatif et DR	Velero, snapshots PVC, dump SGBD

Livable 3 — Votre position d'architecte. en 10 lignes maximum, répondez : « *demain je deviens responsable plateforme dans une boîte qui a 10 services et 30 développeurs. Voici ce que je garde de la chaîne du TP 3, voici ce que je remplace, voici ce que j'ajoute, et pourquoi.* » C'est de l'argumentaire, pas du YAML.

Validation. restituez le livable 3 à voix haute au formateur, sans le relire. C'est un exercice de prise de parole technique.

Boss final (facultatif)

Objectif. vous êtes en astreinte. Le formateur va casser **une seule chose** dans la chaîne progressive delivery + observability. Charge à vous de la diagnostiquer et de la réparer **sans tout détruire**.

Règles du jeu.

1. Vous avez terminé les étapes 0 à 9. À ce stade, votre chaîne tourne : Prometheus scrape, Grafana affiche, Argo Rollouts promeut sur AnalysisTemplate.
2. Vous prévenez le formateur. Il intervient pour casser **un seul élément** (un sélecteur de ServiceMonitor, un seuil d'AnalysisTemplate, une route Alertmanager, un secret Grafana, un ingress, etc.) sans vous dire ce qu'il a fait.
3. Le symptôme apparaît : vous ne voyez plus de trafic dans Prometheus, ou un canary échoue de manière inexplicable, ou une alerte ne déclenche pas, ou un rollback ne se fait pas.
4. Vous diagnostiquez et corrigez **par Git uniquement** (aucun `kubectl` edit pour réparer — c'est tout l'intérêt du GitOps).
5. À la fin, vous expliquez au formateur : *quelle hypothèse vous avez écartée, et pourquoi celle que vous avez retenue collait.*

Critère de réussite. au moins l'une de ces trois conditions :

- vous trouvez et réparez via un commit Git ;
- vous ne trouvez pas mais vous avez bien posé le diagnostic (cause identifiée, plan d'action écrit) ;
- vous ne trouvez pas mais vous avez expliqué proprement comment vous *préviendriez* ce type d'incident à l'avenir.

Note. cet exercice est là pour vous tester en posture d'astreinte. Ce que vous y vivez vaut tout le reste du TP.

Annexe A — Pour aller plus loin

Les sujets ci-dessous ne sont pas traités dans le tronc commun mais prolongent naturellement la chaîne du TP 3. Pour qui finit avant et veut continuer.

Extension	Ce que vous y gagnez
Brancher OpenTelemetry sur l'un des services	Traçabilité distribuée bout-en-bout.
Installer Loki + Promtail	Logs centralisés, corrélés via <i>exemplars</i> aux métriques.
Installer Tempo et brancher Grafana	Voir une trace à partir d'un exemplar dans un panneau de métriques.
Migrer un Rollout en mode <i>experiment</i> avec deux variantes	A/B testing technique.
Installer Chaos Mesh et provoquer un <i>pod kill</i> aléatoire	Mesurer la résilience réelle de votre service.
Installer Kyverno + une policy « <i>no Rollout without AnalysisTemplate</i> »	Forcer la rigueur SRE par politique.
Brancher Argo Rollouts sur un service mesh (Linkerd)	Comprendre le trafic split L7 vs L4.
Implémenter une alerte « <i>error budget brûle plus vite que prévu</i> »	Voir l'aspect <i>multi-window multi-burn-rate</i> des SRE Google.

Annexe B — Pièges fréquents

Symptôme	Cause la plus probable
Prometheus ne voit pas mon service	Label <code>release: <nom></code> manquant sur le Service. Le <code>ServiceMonitor</code> est posé mais Prometheus filtre.
<code>histogram_quantile</code> retourne NaN	Aucun bucket ou pas assez de points dans la fenêtre. Augmentez la durée de scrape ou le trafic de test.
Canary monte à 100 % en quelques secondes	<code>autoPromotionEnabled: true</code> est resté par défaut en blueGreen, ou les <code>pause: {duration: ...}</code> sont en secondes alors que vous attendiez des minutes.
Mon <code>AnalysisTemplate</code> retourne toujours <i>successful</i>	La requête PromQL ne distingue pas canary et stable, vous mesurez la moyenne — qui passe. Filtrez sur <code>rollouts-pod-template-hash</code> ou <code>version</code> .
<code>kubectl argo rollouts get</code> n'existe pas	Plugin pas installé. Voir étape 0.
Alertmanager déclenche en boucle	<code>for absent</code> dans la <code>PrometheusRule</code> , ou <code>repeat_interval</code> trop bas.
Grafana ne montre pas mon dashboard après commit dans Git	Vous l'avez créé en UI mais pas exporté + commité, ou vous n'avez pas configuré le <code>sidecar</code> de provisioning du dashboard.
Mon Rollout reste <code>Progressing</code> indéfiniment	Un <code>AnalysisRun</code> en <code>Inconclusive</code> . Lisez ses mesures : généralement des NaN (pas assez de trafic).
Le <code>previewService</code> du blueGreen est inaccessible	Pas d'Ingress qui pointe dessus. La doc le dit, mais on l'oublie.
Image OCI ARM manquante (Apple Silicon)	Certains charts (versions anciennes) n'ont pas d'image ARM ; bumper la version ou forcer <code>nodeSelector</code> .

Annexe C — Glossaire express

Terme	Définition courte	À ne pas confondre avec...
SLI	Service Level Indicator — un signal mesurable	une simple métrique
SLO	Service Level Objective — un seuil sur un SLI	un SLA (contractuel)
SLA	Service Level Agreement — un engagement contractuel avec pénalité	un SLO (interne)
Error budget	la portion du SLO qu'on peut « brûler » avant d'être en faute	un budget financier
RED method	Rate, Errors, Duration — orientation requêtes	USE method (orientation ressources)
USE method	Utilization, Saturation, Errors — orientation ressources	RED
Histogram (Prometheus)	distribution agrégeable côté requête	Summary (quantiles côté client, non agrégeables)
Recording rule	une requête PromQL pré-calculée à intervalle régulier	une alerte (qui <i>trigger</i>)
Alerting rule	une PromQL qui passe en <i>firing</i> si vraie pendant <i>for</i> : X	une recording rule
Rollout	CRD d'Argo Rollouts qui remplace Deployment	un RollingUpdate natif
Canary	déploiement progressif sur une fraction du trafic	blueGreen (deux stacks complètes en parallèle)
Blue/Green	deux stacks complètes, bascule unique du trafic	canary
AnalysisTemplate	description d'une analyse réutilisable	AnalysisRun (une instance d'analyse)
AnalysisRun	une exécution concrète d'un AnalysisTemplate	AnalysisTemplate (le modèle)
ServiceMonitor	CRD qui dit à Prometheus <i>qui</i> scraper	PodMonitor (scraping de pods directement)
PrometheusRule	CRD qui contient alertes et recording rules	configuration Alertmanager (qui les <i>route</i>)
Traffic split	partage du trafic entre stable et canary	rolling update (juste un remplacement progressif)
DORA metrics	4 indicateurs DevOps : deploy freq, lead time, MTTR, change failure rate	métriques applicatives